Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

**WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI,
INFORMATYKI I INŻYNIERII BIOMEDYCZNEJ**

KATEDRA METROLOGII I ELEKTRONIKI

Sprawozdanie

Wprowadzenie do programowania

Autor: Michał Wojna
Kierunek studiów: Mikroelektronika w Technice i Medycynie
Prowadzący: dr inż. Zbigniew Marszałek

Kraków, 2024

Procesy i przetwarzanie potokowe w GNU/Linux

Michał Wojna, Data: 19.11.2024r. , email: michalwojna@student.agh.edu.pl

Streszczenie—Celem tytułowych ćwiczeń laboratoryjnych jest nauka wyszukiwania danych z użyciem wybranych parametrów, sprawdzanie poboru zasobów komputera przez wybrane procesy, znaczenie priorytetu wykonywanych procesów.

Słowa kluczowe—>, >>, <, <<, cat, cut, head, tail, grep, sed, tr, sort, uniq, wc, xargs, proces1 | proces2 | proces2, 1>plik 2>err, proc1 | tee p1.txt | proces2 > plik.out

I. WSTĘP

System operacyjny Linux różni się znacząco od dobrze znanego i popularnego systemu Windows, zwłaszcza pod względem organizacji ścieżek systemowych czy rozróżniania wielkich i małych liter w nazwach plików i katalogów. Linux należy do systemów z rodziny systemów operacyjnych UNIX.

W rozdziale II przedstawiono rozwiązania serii zadań, wykonanych podczas ćwiczeń laboratoryjnych, w środowisku Linux, wraz z szczegółowym omówieniem i wnioskami z poszczególnych zagadnień związanych z procesami i przetwarzaniem potokowym w systemie Linux z poziomu terminala znakowego.

W rozdziale III znajduje się podsumowanie, w rozdziale IV wykaz oprogramowania. Sprawozdanie kończy rozdział bibliografia.

II. PRZEBIEG ĆWICZENIA

Listing 1. Komenda ps z przełącznikami, wykonana w terminalu znakowym

```
$ ps
    PID TTY          TIME CMD
    329 pts/0    00:00:00 bash
    502 pts/0    00:00:00 ps
$ ps x
    PID TTY      STAT   TIME COMMAND
    329 pts/0    Ss     0:00 -bash
    432 ?        Ss     0:00 /usr/lib/systemd/systemd --user
    433 ?        S      0:00 (sd-pam)
$ ps ax
    PID TTY      STAT   TIME COMMAND
      1 ?        Ss     0:00 /sbin/init
      2 ?        Sl     0:00 /init
      6 ?        Sl     0:00 plan9 --control-socket 6 --log-level 4 –se
$ ps l
F   UID     PID    PPID PRI  NI    VSZ   RSS WCHAN  STAT TTY        TIME COMMAND
4  1000     329     324  20   0   6072  5232 do_wai Ss   pts/0      0:00 -bash
4  1000     446     331  20   0   6072  5132 core_s S+   pts/1      0:00 -bash
$ ps u
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
michal       329  0.0  0.0   6072  5232 pts/0    Ss   09:50   0:00 -bash
michal       446  0.0  0.0   6072  5132 pts/1    S+   09:50   0:00 -bash
```

Komenda ps wyświetla listę aktywnych procesów. Przełącznik x powoduje, że komenda ps wyświetla listę procesów należących do bieżącego użytkownika. Przy użyciu przełącznika a możemy

zobaczyć wszystkie procesy, przełącznik l wyświetla aktywne procesy w formacie długim, który pokazuje nam więcej informacji, przełącznik u wyświetla aktywne procesy w formacie, który jest wygodny dla użytkownika.

Listing 2. Nazwa terminala, struktura procesów

```
$ tty
/dev/pts/0
$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
michal       329     324  0 09:51 pts/0    00:00:00 -bash
michal       655     329  0 10:07 pts/0    00:00:00 ps -f
$ ps 329
    PID TTY      STAT   TIME COMMAND
    329 pts/0    Ss     0:00 -bash
$ ps 1
    PID TTY      STAT   TIME COMMAND
      1 ?        Ss     0:00 /sbin/init
$ pstree
systemd─┬─2*[agetty]
        ├─cron
        ├─dbus-daemon
        ├─init-systemd(Ub─┬─SessionLeader───Relay(329)───bash───pstree
        │                 ├─SessionLeader───Relay(610)───bash
        │                 ├─SessionLeader───Relay(626)───bash
        │                 ├─init───{init}
        │                 ├─login───bash
```

Za pomocą komendy tty możemy sprawdzić nazwę terminala, na którym pracujemy, jeżeli otworzymy kolejne okno terminala, nazwa nowo otworzonego terminala będzie inna. Ps -f pokazuje nam listę procesów z ich numerami PID(Process IDentification i PPID(Parent Process IDentification), za pomocą PPID możemy znaleźć PID procesu macierzystego dla procesu ps. PID jest nadawane w kolejności w jakiej procesy są uruchamiane, więc szukając procesu o PID = 1 znajdziemy przodka wszystkich procesów. Za pomocą komendy pstre możemy zobaczyć graficzne odwzorowanie struktury aktywnych procesów.

Listing 3. Komanda top

```
$ top
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
    670 michal    20   0    9288   5328   3188 R   0.3   0.1   0:00.03
      1 root      20   0   21696  12808   9400 S   0.0   0.2   0:00.73
      2 root      20   0    2616   1440   1320 S   0.0   0.0   0:00.01
$ top
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
    212 root      20   0 2213404  46524  19272 S   0.0   0.6   0:00.77
    249 root      20   0  107000  22584  13248 S   0.0   0.3   0:00.11
     51 root      19  -1   50436  16364  15260 S   0.0   0.2   0:00.47
```

Komanda top wyświetla listę procesów z ich obciążeniami systemu, aktualizującą się co 1s. przy pomocy wciśnięcia shift+c procesy można posortować po ich użyciu procesora, a shift+m sortuje według ilości pamięci ram zajmowanej przez procesy.

Listing 4. Sprawdzanie PID oraz używanie edytora vim

```
$ pgrep bash
329
446
610
626
$ man 7 signal
$ vim
$ pgrep vim
755
$ kill 755
Vim: Caught deadly signal TERM
Vim: preserving files...
Vim: Finished.
Terminated
```

Za pomocą komendy pgrep bash możemy zobaczyć PID naszych wszystkich interpreterów poleceń. Po otwarciu edytora vim, możemy wysyłać do niego sygnały (takie jak np. kill) z innego terminala, należy wtedy znać PID edytora vim, do którego chcemy przesyłać sygnały, który można sprawdzićza pomocą komendy pgrep.

Listing 5. Pkill i killall

```
$ man pkill
$ man killall
```

Zarówno pkill jak i killall jako argument mogą użyć nazwy procesu, ale killall dodatkowo posiada więcej opcji oraz może jakończyć wszystkie instancje danego procesu wraz z jego procesami potomnymi.[1]

Listing 6. Sleep, praca w tle i pierwszym planie

```
$ sleep 15 &
[1] 1042
$ fg
sleep 15
```

Poprzez dodanie po komendzie znaku &, komenda te będzie działać w tle, w tym przypadku komenda sleep zostanie „wywołana” dopiero gdzy przywrócimy ją do trybu pracy w pierwszym planie poprzez komendę fg.

Listing 7. Wstrzymywanie edytora vim oraz jego praca w tle i planie pierwszym

```
$ vim

[1]+  Stopped                 vim
$ vim

[2]+  Stopped                 vim
$ jobs
[1]-  Stopped                 vim
[2]+  Stopped                 vim
$ fg 1
vim

[1]+  Stopped                 vim
$ bg 1
[1]+ vim &

[1]+  Stopped                 vim
$ jobs
[1]+  Stopped                 vim
[2]-  Stopped                 vim
```

Przy użyciu komendy vim dwukrotnie otwieramy i zatrzymujemy (ctrl+z) edytor vim, przy użyciu komendy jobs możemy zauważyć, że oba edytory vim pracują teraz w tle, możemy jeden z nich przywrócić do pracy w trybie pierwszoplanowym przy użyciu komendy fg.

Listing 8. Lista użytkowników i uruchomione przez nich polecenia

```
who
michal   pts/1        2024-11-19 09:50
$ ps -u michal
   PID TTY          TIME CMD
   329 pts/0    00:00:00 bash
   432 ?        00:00:00 systemd
   433 ?        00:00:00 (sd-pam)
   (ucięto)
   856 pts/2    00:00:00 bash
  1106 pts/0    00:00:00 vim
  1108 pts/0    00:00:00 vim
  1113 pts/0    00:00:00 ps
```

Sprawdzamy listę zalogowanych użytkowników komendą who, by zobaczyć uruchomione przez nich polecenia używamy komendy ps z przełącznikiem -u oraz wpisując nazwę użytkownika, który nas interesuje, w tym przypadku jestem jedynym zalogowanym użytkownikiem zalogowanym na moim laptopie.

Listing 9. Status zakończenia procesu

```
$ pgrep
pgrep: no matching criteria specified
Try `pgrep --help' for more information.
$ echo $?
2
$ ps
  PID TTY          TIME CMD
  329 pts/0    00:00:00 bash
 1106 pts/0    00:00:00 vim
 1108 pts/0    00:00:00 vim
 1134 pts/0    00:00:00 ps
$ echo $?
0
```

By sprawdzić status zakończenia programu możemy użyć komendy echo $?, wszystkie wartości poza 0 oznaczają błąd, tutaj została zwrócona wartość 2, ponieważ wywołałem komendę pgrep bez żadnego argumentu. A w drugim przypadku zostało zwrócone 0, ponieważ poprawnie użyłem komendy ps

Listing 10. Obniżanie priorytetu procesu

```
$ find /usr -type f -name "*.py" & renice +19 -p
[3] 1185
$ /usr/lib/ubuntu-advantage/daemon.py
/usr/lib/ubuntu-advantage/convert_list_to_deb822.py
/usr/lib/ubuntu-advantage/upgrade_lts_contract.py
```

Komenda renice pozwala nam obniżyć lub zwiększyć priorytet danego procesu, renice ma zakres od -20 do 19, im mniejszy numer, tym większy priorytet ma dany proces.

Listing 11. Komenda cat

```
$ cp /etc/passwd ~/tekst1
$ ls
Dokumenty  Muzyka  Obrazy  Pobrane  Publiczny  Pulpit  Szablony  tekst1  Wideo
$ cat tekst1 > tekst3
$ cat tekst3
sshd:x:122:65534::/run/sshd:/usr/sbin/nologin
student:x:1001:1001:student,,,,:/home/student:/bin/bash
boss:x:1000:1000:,,,:/home/boss:/bin/bash
$ cat << EOF >> tekst3
1
2
3
4 linia dowolnego tekstu
EOF
4 cat tekst3
boss:x:1000:1000:,,,:/home/boss:/bin/bash
1
2
3
4 linia dowolnego tekstu
```

```
21 $ date > tekst2
22 $ cat tekst2 >> tekst3
23 $ cat tekst3
24 3
25 4 linia dowolnego tekstu
26 śro, 30 paź 2024, 13:32:01 CET
```

Komenda cat pozwala nam wyświetlać zawartość plików tekstowych oraz przekierowywać dane wyjściowe do innych plików za pomocą >, >>, < oraz <<. > i < nadpisują dane a >> i << dodają. EOF jest skrórem End Of File, co w tym przypadku kończy dodawanie tekstu do pliku tekst3.

Listing 12. Użycie komend head, tail oraz użycie pipe

```
1  $ head ~/* -n 5 -q
2  head: błąd czytania '/home/student/Szablony': Jest katalogiem
3  root:x:0:0:root:/root:/bin/bash
4  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
5
6  $ head ~/* -n 5 -q 2>/dev/null
7  root:x:0:0:root:/root:/bin/bash
8  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
9  bin:x:2:2:bin:/bin:/usr/sbin/nologin
10
11 $ head -n 5 /etc/passwd | tail -n3
12 bin:x:2:2:bin:/bin:/usr/sbin/nologin
13 sys:x:3:3:sys:/dev:/usr/sbin/nologin
14 sync:x:4:65534:sync:/bin:/bin/sync
15
16 $  tail -n7 /etc/passwd | head -n3 | tac
17 geoclue:x:120:127::/var/lib/geoclue:/usr/sbin/nologin
18 hplip:x:119:7:HPLIP system user,,,:/var/run/hplip:/bin/false
19 pulse:x:118:125:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
```

Komenda head pozwala nam zmniejszyć ilość wyświetlanych wyników -n oznacza że ograniczamy wyniki do pierwszych x linijek (w linii 1 do 5). Tail działa na tej samej zasadzie ale ograniczamy ilość ostatnich x linijek. Pipe (|) pozwala nam przekierować wynik jednego polecenia do następnego.

Listing 13. UZycie komendy tr oraz wc

```
1  $ cat /etc/passwd | tr -d '\n'
2  root:x:0:0:root:/root:/bin/bashdaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologinbin:
3  x:2:2:bin:/bin:/usr/sbin/nologinsys:x:3:3:sys:/dev:/usr/sbin/nologinsync:x:4:65534:
4  sync:/bin:/bin/syncgames:x:5:60:games:/usr/games:/usr/sbin/nologinman:x:6:12:m
5  an:/var/cache/man:/usr/sbin/nologinlp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologinmail
6  :x:8:8:mail:/var/mail:/us
7
8  $ mv tekst3 teksttest
9  $ ls
10 Dokumenty  Muzyka  Obrazy  Pobrane  Publiczny  Pulpit  Szablony  tekst1  tekst2  teksttest
11 Wideo
12 $ cat teksttest | tr ' ' '\n' > tekst3
13 $ ls
14  Dokumenty  '\n'    Pobrane    Pulpit    tekst1   tekst3     Wideo
```

```
 Muzyka    Obrazy   Publiczny   Szablony   tekst2   teksttest
$ cat tekst3
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
$ find /etc/ -type f 2>/dev/null | wc -l
1953
$ head -n3 /etc/passwd | tr -d '\n' |  wc -c
114
$ ls . | tr '[:lower:]' '[:upper:]'
DOKUMENTY
MUZYKA
```

Komenda tr służy do usunięcia wybranych znaków (-d) bądź gdy się powtarzają, sprowadzenie ich do jednego (-s). Komenda wc liczy linijki (-l), słowa (c), lub słowa (w)

Listing 14. Sortowanie oraz Komenda grep

```
$ ls -al | grep ^- | tr -s ' ' | cut -f1,5,9 -d ' '
-rw------- 89 .bash_history
-rw-r--r-- 220 .bash_logout
-rw-r--r-- 3771 .bashrc
$ ls -al | grep ^- | tr -s ' ' | cut -f5,9 -d ' ' | sort -n | cut -f2 -d ' '
\n
.gtkrc-2.0
.dmrc
$ cat /etc/passwd | sort -k3 -nrt:
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
student:x:1001:1001:student,,,,:/home/student:/bin/bash
boss:x:1000:1000:,,,:/home/boss:/bin/bash
$ cat /etc/passwd | sort -k4 -k3 -nt:
sync:x:4:65534:sync:/bin:/bin/sync
_apt:x:104:65534::/nonexistent:/usr/sbin/nologin
kernoops:x:106:65534:Kernel Oops Tracking Daemon,,,:/:/usr/sbin/nologin
```

Grep ZMIENNA wyświetla tylko wyniki, które w swojej nazwie zawierają ZMIENNĄ, jeżeli użyjemy grepa z opcją -v, wyświetlone zostaną wyniki nie zawierające ZMIENNEJ. Sort można używać z przełącznikami (domyślne jest sortowanie numeryczne), -r sortuje odwrotnie, -b ignoruje spacje na początku, kLICZBA przy sortowaniu bierze pod uwagę LICZBA kolumnę, -t pozwala zmienić somyślny separator kolumn.

Listing 15. Liczenie plików i statystyka

```
$ find / -type f -user student 2>/dev/null | wc -l
58241
$ ls -al | cut -f1 -d' ' | sort | uniq -c
      4 drwx------
     16 drwxr-xr-x
      1 razem
      5 -rw-------
      7 -rw-r--r--
      4 -rw-rw-r--
```

2>/dev/null powoduje, że wszytkie komunikaty o błędach zostają przekierowane do folderu /dev/null (kosz bez dna). Uniq -c liczy powtarzające się wartości, z czego wiemy ile występuje danego typu praw dostępu.

Listing 16. Sortowanie po rozmiarze 3 najmniejszych plików z katalogu domowego.

```
1  $ ls -lp | tr -s ' ' | grep -v /| cut -f5,9 -d ' ' | sort -n | cut -f2 -d ' ' | sort | head -n4 | tail -n3
2  li
3  ls
4  plik3
```

Przełącznik -p na końcu każdego katalogu stawia /, po czym wykluczamy z wyszukiwanych danych katalogi nie biorąc ich pod uwagę (grep -v /).

Listing 17. 3 użytkowników z największą ilością procesów

```
1  $ ps -ef | tr -s ' ' | cut -f1 -d ' ' | sort | uniq -c | sort -r | head -n3
2      18 root
3      11 michal
4       2 systemd+
```

Opcja -e przy ls powoduje wyświetlanie nazwy właściciela procesu.

Listing 18. Pliki w 3 największych podkatalogach katalogu domowego

```
1  $ ls -lp | tr -s ' ' | grep /| cut -f5,9 -d ' ' | sort -nr | cut -f2 -d ' ' | head -n 3 | xargs ls
2  snap/:
3  htop
4  src1/:
5  wo/:
```

Xargs ls bierze wyniki poprzedniej komendy i wyświetla ich zawartość (ls).

Listing 19. Użytkownicy nie używający interpretera basha

```
1  $ cat /etc/passwd | cut -d: -f1,7 | grep -v bash | sort -ut: -k2 | tail -n3
2  dhcpcd:/bin/false
3  sync:/bin/sync
4  daemon:/usr/sbin/nologin
```

Informacja o używanym przez danego użytkownika interpreterze znajduje się w folderze /etc/passwd. Sort -u sortuje alfabetycznie w jednej kolumnie.

Listing 20. Statystyka użycia komend

```
1  $ history | tr -s ' ' | cut -f3 -d ' ' | sort | uniq -c | sort -r
2      244 ls
3      113 ps
4      101 cd
5       48 find
6       40 jobs
```

Komenda history wyświetla wszystkie użyte w aktywnej sesji polecenia wydane przez użytkownika.

Listing 21. Utworzenie 5 katalogów wewnątrz nowo utworzonego katalogu

```
1 $ mkdir users; cd users; mkdir 1; mkdir 2; mkdir 3; mkdir 4; mkdir 5
2 $ ls
3 1  2  3  4  5
```

Znak ; pozwala na wydanie wielu komend na raz w danej kolejności.

Listing 22. Kto jest zalogowany więcej niż 1 raz.

```
1 $ who | tr -s ' ' | cut -f1 -d ' ' | uniq -d
```

Uniq -d ignoruje pojedyncze dane. Brak wyników ponieważ pracuje na własnym laptopie.

Listing 23. Wyświetlenie posortowane plików nagłówkowych znajdujących się w katalogu /usr/include/

```
1 $ find /usr/include/ -type f -name *.h -exec grep ^#include {} \; | tr -s ' ' | cut -f2 -d ' ' | sort
2 <asm/types.h>
```

Pliki nagłówkowe mają rozszerzenie .h

Listing 24.

```
1 $ nice -n 19 find -type f -name '*user*' | 2>/dev/null | 1>wyniki.txt
2 $ echo $?
3 0
4 $ cat wyniki.txt
```

Plik jest pusty, ale komenda z linii 1 zakończyła się sukcesem (echo $?)

III. PODSUMOWANIE

Wykonane ćwiczenia pozwoliły na opanowanie podstawowych zagadnień związanych z wyszukiwaniem danych z użyciem parametrów, oraz sortowaniem ich by były przejrzyste. Podstawowa umiejętność wyszukiwaniem danych z użyciem parametrów, oraz sortowaniem ich by były przejrzyste z poziomu terminala znakowego przydaje się szczególnie wówczas, gdy mamy do czynienia z tzw. systemami embedded [5,6] przeznaczonymi na małe urządzenia pozbawione interfejsu graficznego.

IV. WYKAZ OPROGRAMOWANIA

- System operacyjny 5.15.167.4-microsoft-standard-WSL2

Bibliografia

[1] https://pl.linux-console.net/?p=1976